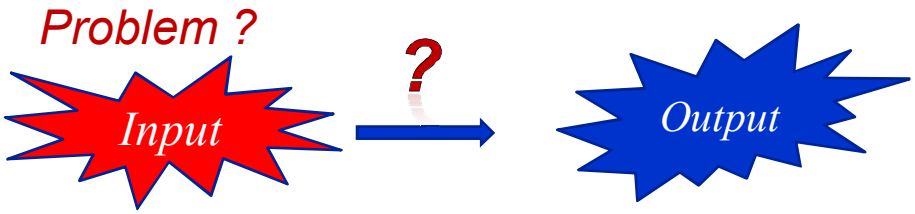
Algorithms

Introduction,
Correctness Proof,
Techniques for Analysis of Algorithms

Dr. Md. Abul Kashem Mia, Professor, CSE Dept, BUET

What is an Algorithm?

An algorithm is a sequence of instructions that one must perform in order to solve a well-formulated problem.



Algorithms did not start with computers; they have been with us from ancient times; nor they are limited to computer science.

What is an Algorithm?

- An algorithm is a sequence of computational steps that transform the input into the output.
- An algorithm is a tool for solving a well-specified computational problem.
  - An algorithm is said to be correct if, for every input instance, it halts with the correct output
  - An incorrect algorithm might not halt at all on some input instances, or it might halt with other than the desired output.

Why do we study algorithms?

Why do we *need a good* algorithm?

Computing Fibonacci Numbers

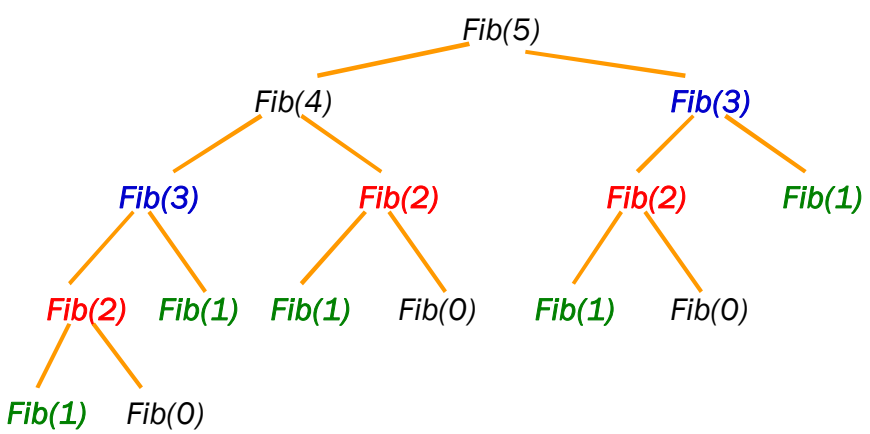
- Fibonacci numbers:
  - $F_0 = 0$
  - $F_1 = 1$
  - $F_n = F_{n-1} + F_{n-2}$ for $n > 1$

  Sequence is 0, 1, 1, 2, 3, 5, 8, 13, …
- Fibonacci Problem: Calculate the *n*th Fibonacci number.
- Obvious recursive algorithm (inefficient):

```
Recursive_Fibonacci (n):
    if n = 0 or 1 then
        return n
    else
        a ← Recursive_Fibonacci(n – 1)
        b ← Recursive_Fibonacci(n – 2)
    return a + b
```

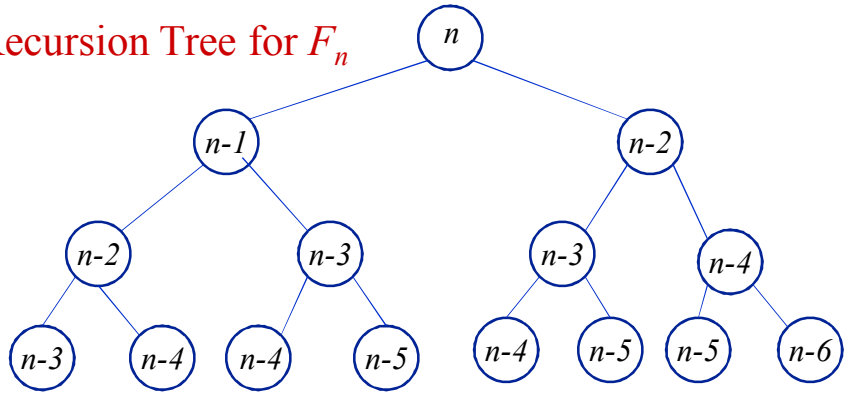

Computing Fibonacci Numbers

Recursion Tree for F_5



Computing Fibonacci Numbers

Recursion Tree for F_n



- *As the tree grows larger, the number of duplicate vertices increases exponentially 2^i-2 at level i, while the number of regular vertices increases linearly (2 per level).*
- *Wasteful approach - repeat work unnecessarily*
  - *Running time: $O(2^n)$*

Computing Fibonacci Numbers

```
F[0] := 0; F[1] := 1;
for i := 2 to n do
    F[i] := F[i – 1] + F[i – 2]
return F[n]
```

- *Optimal approach – no repetition of work*
  - *Running time: $O(n)$*
  - *Time reduced from exponential to linear*

Analysis of Algorithms

- *Correctness of an Algorithm*
- *Performance of an Algorithm*

Correctness of an Algorithm

A correct algorithm/program yields the correct result for all legal input values.

?

Proving the correctness of an algorithm is not at all an easy task. It consists of two steps

1) Proving that the algorithm will always terminate.
2) Proving that it will always produce correct result.

Correctness of an Algorithm

Linear Search Algorithm

Let $X = [x_1, x_2, \ldots, x_n]$ be an unordered list of n distinct items. We would like to search the list for a specific item, called *key*. If *key* exists, in the list, the algorithm should return the location, $1 \le i \le n$, of the *key*, else it returns 0.

Algorithm linear_search(X, n, key, location)

0. ***begin***
1. $x_0 \leftarrow key$
2. $i \leftarrow n$
3. *while* $x_i \neq key$ *do*
4. $i \leftarrow i - 1$
5. $location \leftarrow i$
6. ***end***

Correctness of an Algorithm

Theorem. Algorithm linear_search works correctly for every $n \ge 0$.

Proof. Let $P(n)$: Algorithm linear_search returns the correct location for every list of size $n \ge 0$.

Basis: When $n = 0$, the while loop is skipped. The algorithm returns the value 0 in location by line 5, which is correct. Thus $P(0)$ is correct.

Induction hypothesis: Assume that $P(k)$ is true for an arbitrary integer $k \ge 0$; that is, the algorithm works when the list contains k items.

Induction step: We will show that $P(k + 1)$ is true; that is, the algorithm works when the list contains $k + 1$ items.

0. ***begin***
1. $x_0 \leftarrow key$
2. $i \leftarrow n$
3. *while* $x_i \neq key$ *do*
4. $i \leftarrow i - 1$
5. $location \leftarrow i$
6. ***end***

Correctness of an Algorithm

Induction step: We will show that $P(k + 1)$ is true; that is, the algorithm works when the list contains $k + 1$ items.

We have two cases:

Case 1. If $x_{k+1} = \text{key}$ in line 3, the while loop will not be entered and the algorithm returns the correct value $k + 1$ for location in line 5.

Case 2. If $x_{k+1} \neq \text{key}$, then $i = k$ at the end of first iteration. This restricts us to a sublist with k elements. By induction hypothesis, the algorithm works correctly for such a list.

In both cases, $P(k + 1)$ holds.

Thus $P(n)$ is true for $n \geq 0$.

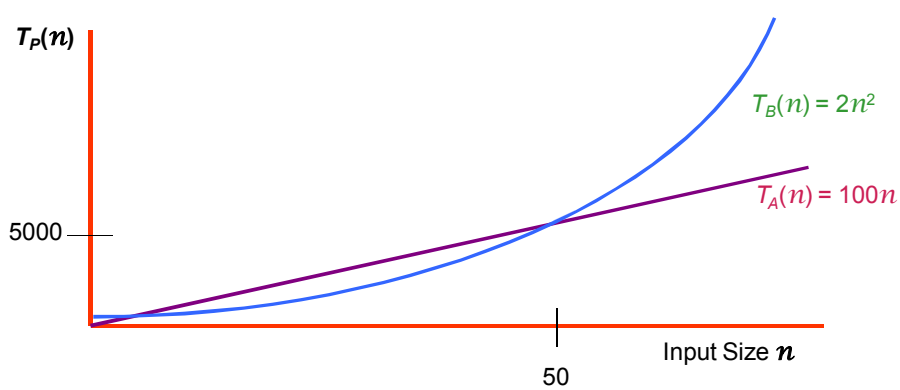
0. ***begin***
1. $x_0 \leftarrow key$
2. $i \leftarrow n$
3. *while* $x_i \neq key$ *do*
4. $\quad i \leftarrow i - 1$
5. $location \leftarrow i$
6. ***end***

Performance of an Algorithm

For large input sizes, constant terms are insignificant

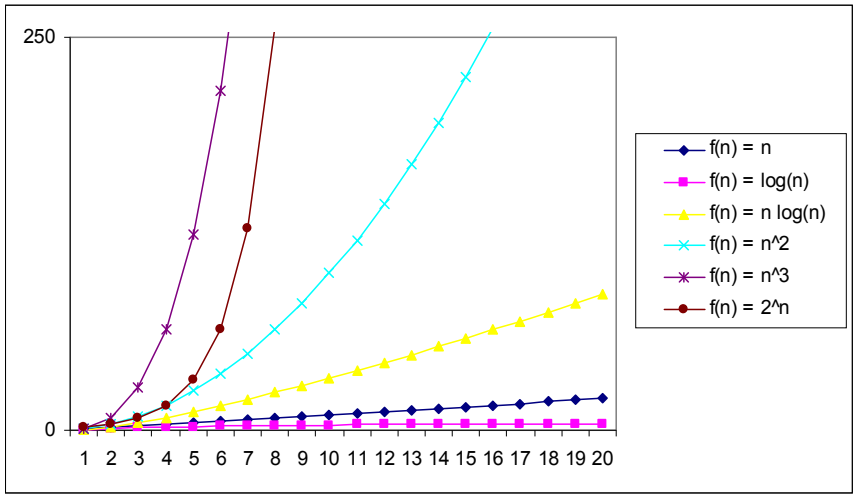
Program A with running time $T_A(n) = 100n$

Program B with running time $T_B(n) = 2n^2$



Performance of an Algorithm

Growth of functions



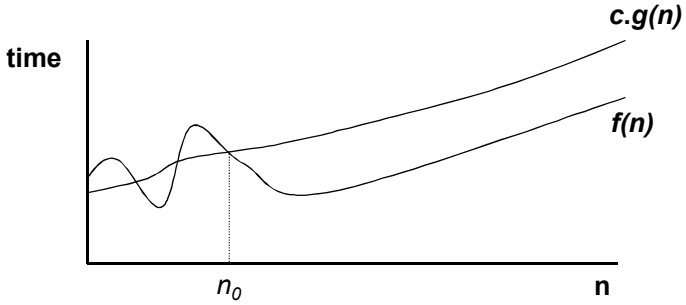
Performance of an Algorithm

The Big-Oh Notation

Growth of functions

Let f, g: **N** → **R**. Then $f(n)$ is of order at most $g(n)$, if there exist a positive constant c and a positive integer n_0 such that $|f(n)| \leq c|g(n)|$ for every $n \geq n_0$.

In symbol we write $f(n) = O(g(n))$.



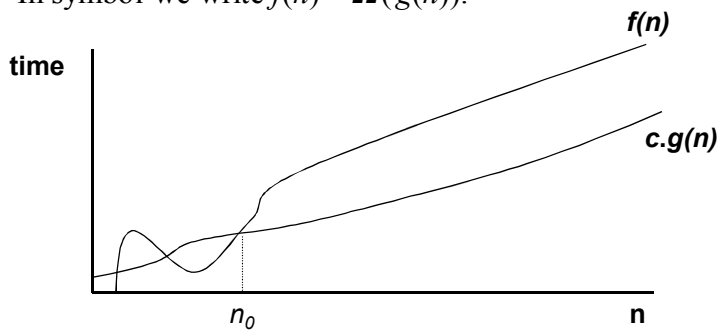
Performance of an Algorithm

The Big-Omega Notation

Growth of functions

Let f, g: **N** → **R**. Then $f(n)$ is of order at most $g(n)$, if there exist a positive constant c and a positive integer n_0 such that $|f(n)| \geq c|g(n)|$ for every $n \geq n_0$.

In symbol we write $f(n) = \Omega(g(n))$.



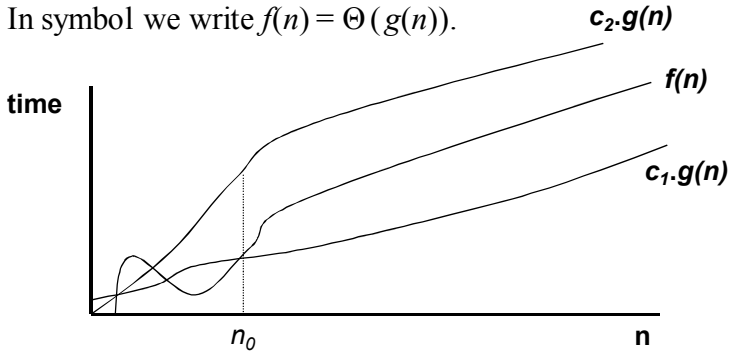
Performance of an Algorithm

The Big-Theta Notation

Growth of functions

Let f, g: **N** → **R** such that $f(n) = O(g(n)$ and $f(n) = \Omega(g(n))$. Then $f(n) = \Theta(g(n))$.

In symbol we write $f(n) = \Theta(g(n))$.



Performance of an Algorithm

A Survey of Common Running Times

- Computing the maximum: $O(n)$
- Merging two sorted Lists: $O(n)$
- Sorting: $O(n \log n)$
- Finding closest pair of points in 2D: $O(n^2)$
- Finding two disjoint sets: $O(n^3)$
- Independent set of size k: $O(n^k)$
- Independent set of maximum size: $O(2^n)$

Algorithm Design Techniques

- Exhaustive Search
- Branch and Bound
- Greedy Method
- Divide and Conquer
- Dynamic Programming
- Machine Learning
- Randomized Algorithm
- Online algorithm